Finding What Changed

A difference-imaging pipeline for the SPHEREx Quick-Release data

SPHERExLens • 23-slide specification and complete script
full route 21:40 • 15-minute route 15:45 • questions after

Evan Lim • November 2026

Deck map and production rules

23 slides on 25 pages. This page is the specification; pages 2–24 are the slides, one page each: the slide's own bullets, its figure brief, the figure itself, then the word-for-word script. Page 25 is a backup page with the five questions you are most likely to be asked and a pre-flight checklist.

The full route runs **21:40**. The slides marked ✓ in the last column form a **15:45** route: seven slides dropped, argument intact. **The live demonstration is slide 22** (1:30) and uses the browser app (`site/index.html` or the deployed Netlify/Cloud Run app) already open in a second tab at 100% zoom, with Barnard's Star loaded – plus Live IRSA mode if showing on-demand fetching.

Every code metric and detection count in this deck is verified directly against the current runtime modules (`step1`–`step7`, support modules, `app.py`, `static/`) and the live Cloud Run snapshot API (`/api/targets`, `/api/dashboard`, `/api/recovery`, `/api/snapshot-history`).

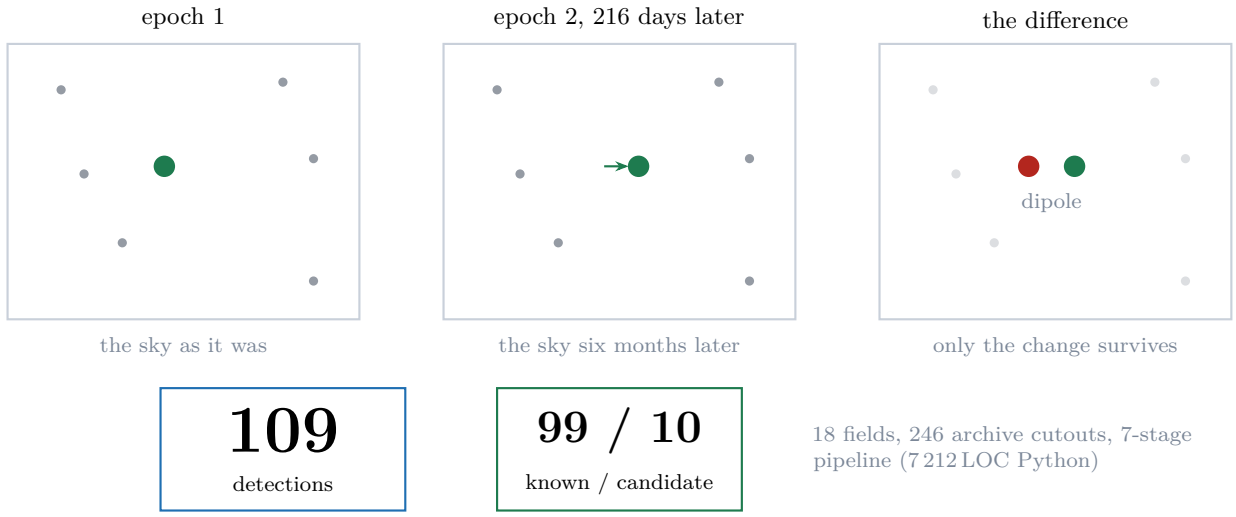
Page	Slide	Dominant visual	Time	15 min
2	Title and the one-number answer	two epochs → difference, hero numbers	0:40	✓
3	Why difference imaging at all	three panels: visit 1, visit 2, difference	0:55	✓
4	The problem statement	pixels → candidates → verdicts	1:00	✓
5	Focus and research questions	four question cards	0:45	–
6	Prior art I: the subtraction lineage	timeline with four method boxes	1:00	✓
7	Prior art II: brokers and the gap	detect–veto–human loop, gap bars	0:55	✓
8	SPHEREx: instrument and data	focal plane, 102-band axis, 6 HDUs	0:55	✓
9	Archive mechanics and field sample	sky map of 18 fields, fetch path	0:55	–
10	Architecture: seven stages	pipeline flow, infra, line counts	0:55	✓
11	Theory I: masking and the co-add	14 bad bits, matched mask, $1/V$ stack	0:55	✓
12	Theory II: ZOGY	signal-flow diagram, null histogram	1:15	✓
13	Theory III: detection statistics	2-tailed false-positive budget, log scale	0:55	–
14	Theory IV: dipole versus monopole	geometry of a moved source	0:50	–
15	Theory V: triage	epoch propagation, 10–120″ cone, 6 classes	0:55	✓
16	Theory VI: spectral diagnosis	CH_4, H_2O, NIR slope + channel gate	0:45	–
17	Validation: injection–recovery	measured 24-cell matrix, noise gap	0:55	✓
18	Implementation I: Python & backend	LOC by module (7 212 total), backend flow	1:05	✓
19	Implementation II: architecture & hosting	Cloud Run split, Netlify, GCS snapshot chain	0:55	✓
20	Implementation III: frontend & UI/UX	wireframe, canvas, snapshot history replay	0:45	–
21	Results I: eighteen fields	stacked detections, 109 across 18 fields	1:05	✓
22	Results II: the candidates	scatter + epoch-propagation fix	1:00	–
23	**Live demonstration**	current HDS UI map, snapshot replay, 6 moves	1:30	✓
24	Conclusion, contributions, Q&A	three pillars, references	0:50	✓
Full route			**21:40**	
15-minute route (16 of 23 slides)			**15:45**	

Slide 1 — Title and the one-number answer 0:40

On the slide.

- SPHERExLens: finding what changed in NASA's SPHEREx all-sky survey
- 18 fields • 246 archive cutouts • 109 detections, 99 explained (91%)
- A codebase you can inspect: seven stages, 7 212 runtime Python lines, one browser app
- The honest number: two fields produce no detections (one quiet nebula, one band-mismatch), and the system says so

Figure / diagram. Two epoch panels of the same star field feeding a difference panel, with the two headline numbers set large beside them.



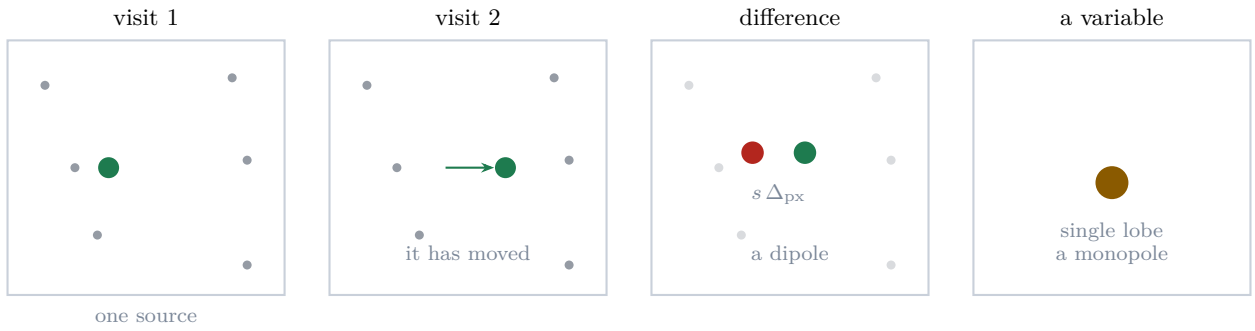
Spoken script. Good morning. NASA's SPHEREx satellite photographs the whole sky in a hundred and two infrared colours, and it does it again roughly every six months. My question is the oldest one in time-domain astronomy, asked of this new data: looking at multiple visits to the same patch of sky, can we say what changed – and can we be honest about what we cannot tell? Eighteen fields, two hundred and forty-six archive cutouts across all six detectors, one hundred and nine detections, of which ninety-nine – ninety-one percent – turn out to match already catalogued or artefact-explained sources. Most of the next twenty minutes is about how the seven-stage pipeline works and why each safeguard is built the way it is.

Slide 2 — Why difference imaging at all 0:55

On the slide.

- One image shows what is there; a pair shows what is happening
- Subtract two visits: the constant sky cancels, the change survives
- Two shapes of change: a dipole (moved) and a monopole (brightened)
- A difference image is still not a discovery – systematics dominate it

Figure / diagram. Three large panels – visit 1, visit 2, the difference – plus a fourth small panel for a variable, and a labelled haze for the systematics that survive subtraction.



what the difference image also contains, and why it is not a discovery: PSF mismatch between visits, detector state, astrometric shifts. These survive the subtraction as a grey haze, and they outnumber the real changes.

Spoken script. Why bother with the second photograph at all? Because a single image tells you what is there, and a pair tells you what is happening. Take the same patch of sky, subtract one visit from the other, and the constant sky cancels to grey. What survives is change, and change has a shape. If a star moved you get a positive lobe where it is now and a negative lobe where it was: a dipole, and the separation divided by the time baseline is a proper motion. If a star simply brightened you get one lobe: a monopole.

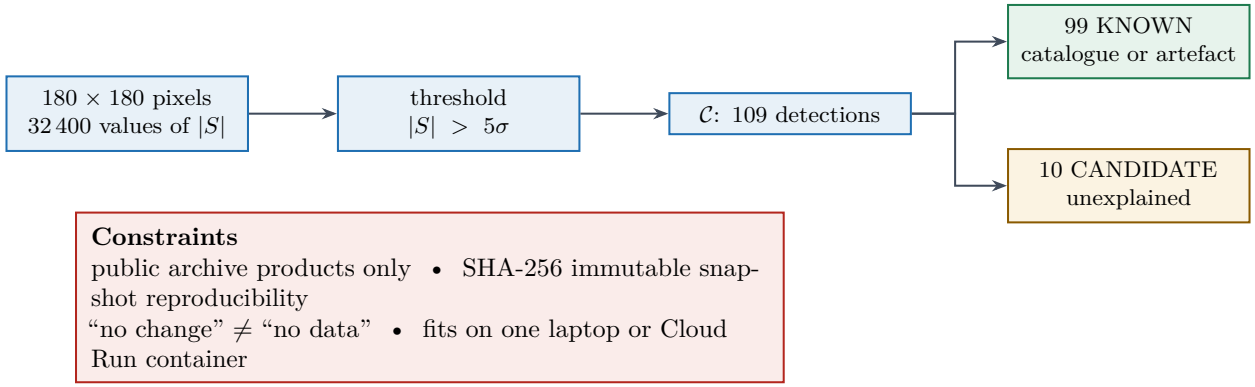
That is the whole physics, and it is fifty years old. The reason this is still a research problem is the grey haze labelled under the third panel: what survives subtraction is dominated by the point-spread function changing between visits, by the detector warming up, by the astrometric solution shifting. Almost none of those pixels are sky. So the work is not the subtraction – it is deciding which of those blobs you are allowed to believe.

Slide 3 — The problem statement 1:00

On the slide.

- Input: two visits to a $9'$ field, each with a variance plane and a quality bitmask
- Output: the set $\mathcal{C} = \{\mathbf{x} : |S(\mathbf{x})| > \tau\}$, with a position, a significance, a morphology and an *identity verdict*
- Constraints: public data only; every number reproducible; "no change" must be distinguishable from "no data"; serverless Cloud Run or laptop scale

Figure / diagram. A left-to-right flow in set notation: 32 400 pixels, a threshold, the 109 detections, then the triage split into 99 known and 10 candidate, with the four constraints in a box underneath.



Spoken script. Let me state the problem precisely, because the precision is the point. Input: two or more visits to a nine-arcminute field, each with a variance plane and a quality bitmask. Task: build a statistic, threshold it, and so obtain the set of sky positions where the flux distribution changed – that is the set C on the slide – and attach to each element four things: a sky coordinate, a significance, a morphology, and an identity verdict. Identity is not optional, because a blob is only interesting if no catalogue or instrumental artefact already explains it.

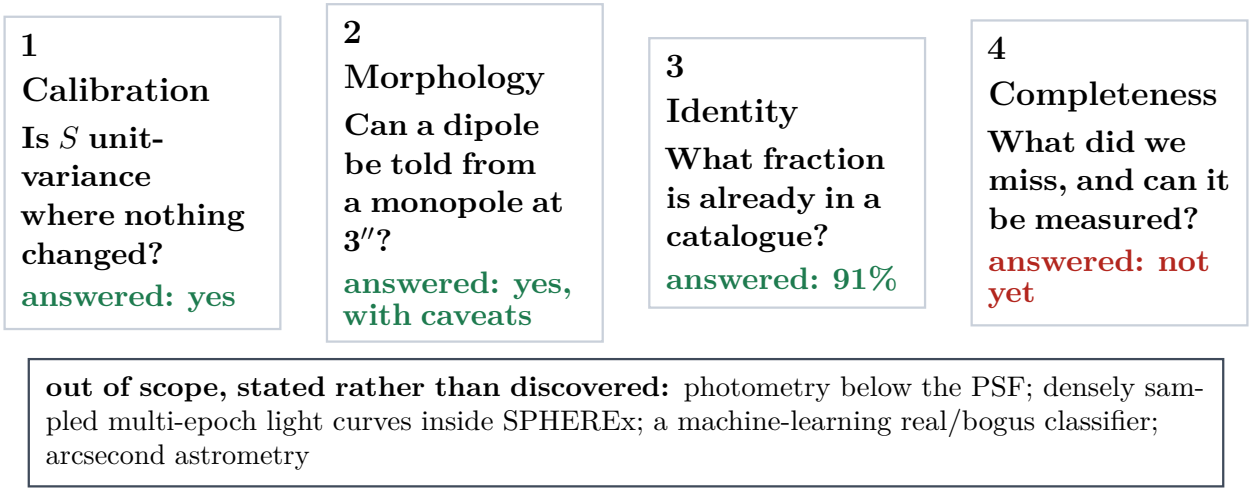
Four constraints shaped every design decision. Only public archive products. Every snapshot reproducible and content-addressed. The system must distinguish "no change" from "no data" or mismatched bands – that is why Sgr A* is explicitly refused when epochs share no detector. And the whole thing runs on one laptop or a single Cloud Run container. The numbers follow from that: thirty-two thousand four hundred pixels per map in, one hundred and nine detections out, ninety-nine explained, ten not.

Slide 4 — Focus and research questions 0:45

On the slide.

- RQ1: is the significance map calibrated on co-added SPHEREx images?
- RQ2: does dipole/monopole morphology separate movers from variables at 3″ sampling?
- RQ3: how much of the candidate list do Gaia, WISE and VSX explain away?
- RQ4: what does injection–recovery say – and if it says "nothing", how is that reported?

Figure / diagram. Four numbered question cards in a row, each with its one-line answer status underneath, and a footer strip listing what is explicitly out of scope.



Spoken script. I want to be explicit about what is and is not in scope, because a small project like this either draws its boundary honestly or it oversells. Four questions, and the answers are on the cards. One: is the statistic actually calibrated, so that five sigma means what it says. Two: can the morphology test tell a moved star from a flickering one at three-arcsecond sampling. Three: how much of the candidate list is already explained by Gaia, WISE, VSX, NEOWISE or mask/bright-star artefacts – ninety-one percent. Four: what did we miss, and here the honest answer is "not yet known", which I will come back to on the validation slide.

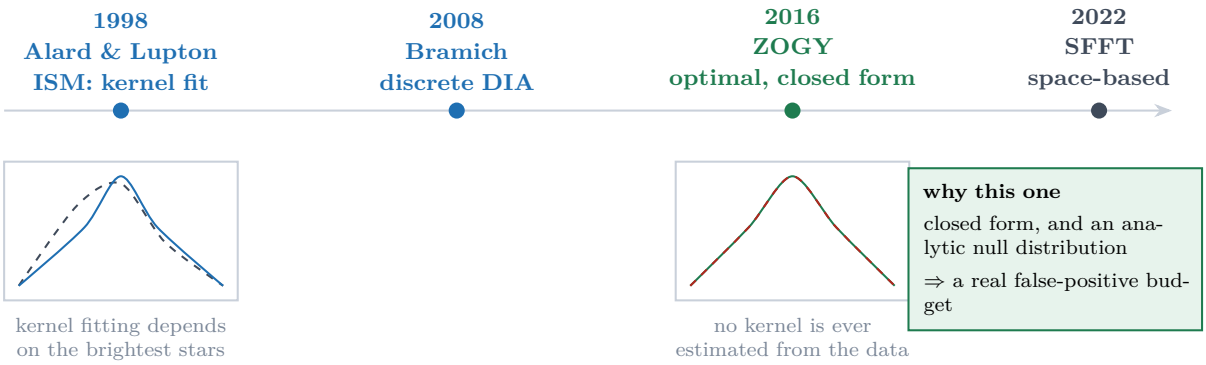
And the footer strip is deliberate: no PSF-level photometry, no dense SPHEREx light curves, no machine-learning real/bogus classifier, no arcsecond astrometry. Stated up front rather than discovered by a reviewer.

Slide 5 — Prior art I: the subtraction lineage 1:00

On the slide.

- 1998 Alard & Lupton: fit a convolution kernel so that both images share a PSF (ISM)
- 2008 Bramich: a discrete per-pixel kernel, well posed in sparse fields
- 2016 ZOGY: no fitting at all; the optimal difference and a unit-variance significance map
- 2022 SFFT: the same mathematics, better behaved for undersampled space PSFs

Figure / diagram. A timeline with the four milestones, and under two of them a small sketch contrasting "fit until the curves agree" with "convolve both to a common PSF".



Spoken script. Subtraction was solved in stages, and the stages matter for what I could assume. In 1998 Alard and Lupton fitted a convolution kernel that maps one image onto the other so that both end up with the same point-spread function – that is what made the microlensing surveys possible. It works, but the fit is dominated by the brightest stars in the field, which is exactly what you cannot rely on in a nebula or a crowded field. In 2008 Bramich made the kernel per-pixel and well posed in sparse fields; you trade noise for stability.

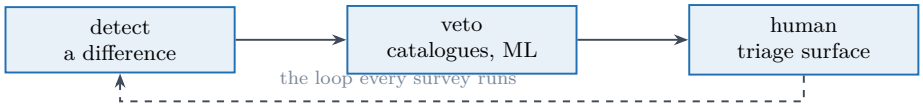
Then in 2016 Zackay, Ofek and Gal-Yam did something conceptually different: instead of fitting anything they derived the statistically optimal difference image in Fourier space, and with it a significance map with unit variance by construction wherever the sky did not change. SFFT in 2022 is the modern reformulation and it is what Rubin will use. I chose ZOGY for the reason on the right of this slide: because the null distribution is analytic, the false-positive budget can be computed rather than measured, and that is what lets me defend a five-sigma threshold later.

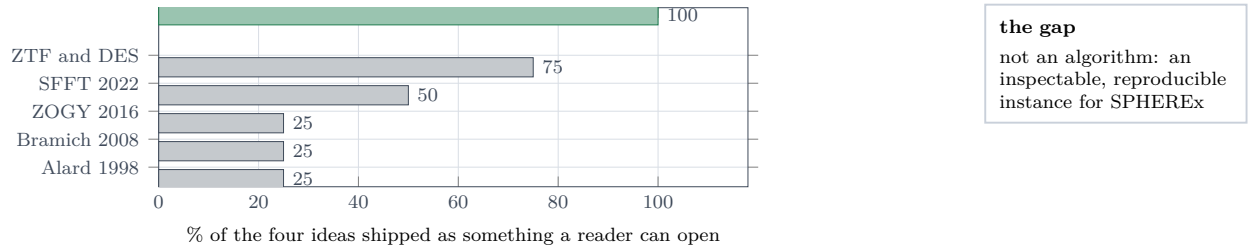
Slide 6 — Prior art II: brokers, and the gap this fills 0:55

On the slide.

- Every operational survey runs the same four ideas: template, subtract, veto, triage
- Brokers (ANTARES, ALeRCE, Lasair, Fink) industrialise the veto with machine learning
- For SPHEREx there is no published open end-to-end difference-imaging artefact
- The gap is not an algorithm – it is an inspectable, reproducible instance

Figure / diagram. The detect–veto–human loop on the left, and on the right a bar chart of how much of those four ideas each body of work actually ships as something a reader can open.





Spoken script. Everything since the 1990s runs the same loop, and it is worth naming as a loop: you detect against a template, you veto with catalogues and increasingly with machine learning, and you put what survives in front of a human. The big surveys industrialised each step – ZTF, ATLAS, DES, and now Rubin, with brokers called ANTARES, ALeRCE, Lasair and Fink doing the vetting. They are enormously more capable than anything I built.

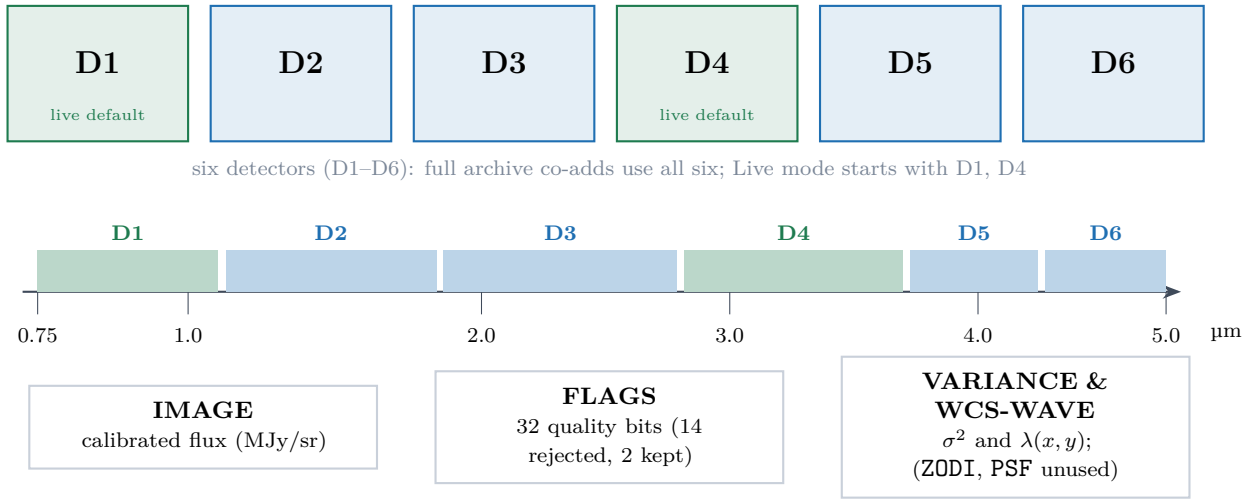
So the gap is not algorithmic. The bars on the right are my own assessment of how much of that four-part pattern each line of work ships as something a reader can actually inspect: the classical papers give equations and sometimes a reference code; the surveys give a pipeline you cannot open; the brokers give a service. What does not exist, as far as I could find, is a small end-to-end artefact for SPHEREx specifically, where you can follow one pixel from a NASA archive file to a verdict on a web page, including the parts that fail. That is what this is.

Slide 7 — SPHEREx: the instrument and the data products 0:55

On the slide.

- One telescope, six detectors, 102 bands from 0.75 to 5.0 µm
- Six FITS extensions per cutout: we use `IMAGE`, `FLAGS` (32 bits), `VARIANCE`, and `WCS-WAVE`
- Full snapshots co-add all six arrays (`D1-D6`); Live IRSA mode defaults to `D1, D4` with 6-band fallback
- Six-month cadence ⇒ what survives subtraction is astrophysical, not asteroids

Figure / diagram. The six-detector focal plane above a wavelength axis (`D1-D6` full coverage and `D1, D4` fast-live default highlighted), with the four HDU extensions used downstream underneath.



Spoken script. A word on the instrument, because the shape of the data drives everything downstream. SPHEREx is a small telescope with six detectors that sweeps the sky and, unusually, takes a low-resolution spectrum at every position: a hundred and two bands between zero-point-seven-five and five microns. Every Level-2 file actually holds six FITS extensions, of which four drive our pipeline: `IMAGE`, `FLAGS`, `VARIANCE`, and `WCS-WAVE` – giving calibrated flux, a thirty-two-bit quality mask, per-pixel variance, and a lookup table from pixel to wavelength. Scheduled snapshots co-add all six arrays, while Live IRSA mode starts with `D1` and `D4` for speed and falls back to all six whenever coverage requires it.

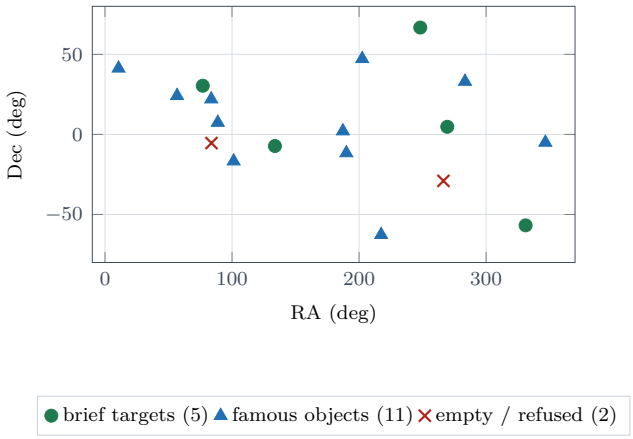
Two consequences. First, the cadence is about six months, so fast solar-system objects have left the field entirely; what survives subtraction is genuine variability or high proper motion. Second, because every pixel carries its wavelength, we can build an infrared SED without follow-up – which is what slide fifteen is about.

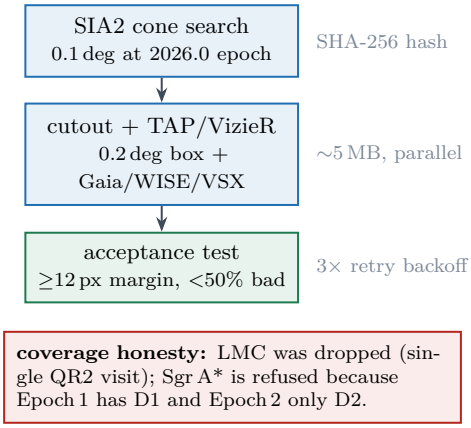
Slide 8 — The archive mechanics, and the eighteen fields 0:55

On the slide.

- Two calls: SIA2 cone search (fingerprint-hashed) → `0.2 deg` cutout service (`INGEST_EPOCH=2026.0`)
- ∼5 MB sections of 70 MB parent frames, plus parallel TAP/VizieR queries for Gaia DR3, AllWISE, VSX
- Acceptance: ≥12 px valid margin inside the cutout, and under 50% of pixels flagged
- 18 fields, 246 cutouts across `D1-D6`: two empty (M42 null, Sgr A* no shared band); LMC dropped

Figure / diagram. Left: the sky distribution of all eighteen fields. Right: the two-step fetch path with its cost annotated, and the acceptance test at the end.





Spoken script. Getting the data takes two archive calls, and high-proper-motion stars are propagated to epoch 2026.0 first so the cone search centres on where the star is now. A SIA2 search lists every Level-2 product and hashes the product list with SHA-256 so scheduled refreshes skip unchanged fields. Then the cutout service delivers five-megabyte sections of seventy-megabyte frames while Gaia DR3, AllWISE, and VSX are queried in parallel via TAP and VizieR mirrors. Every cutout passes an acceptance check – at least twelve pixels of valid margin and fewer than half flagged – before it enters the stack.

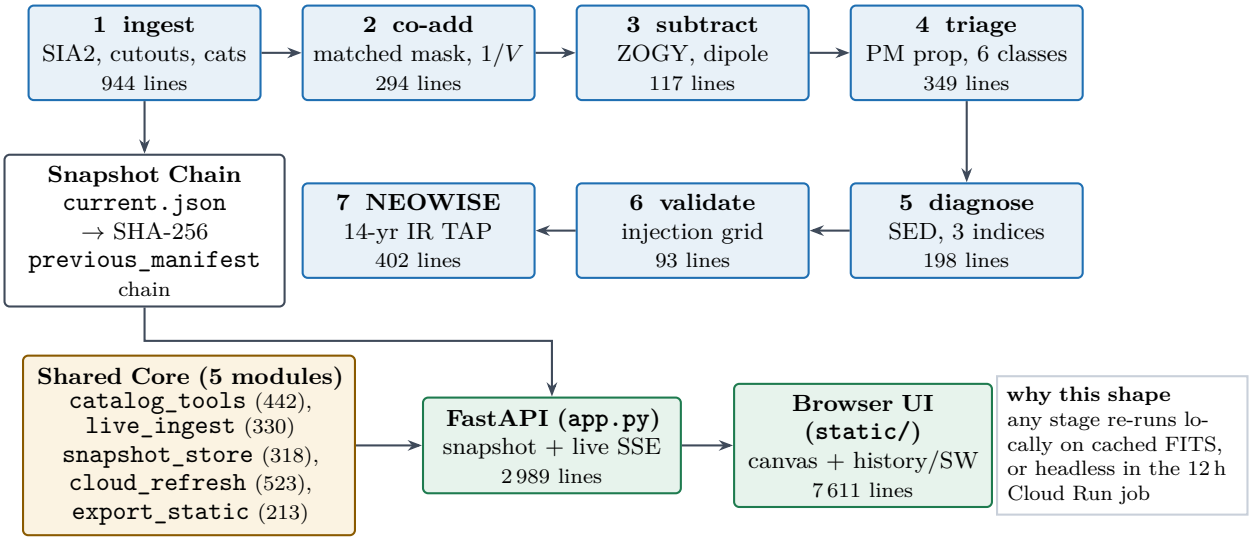
The map shows the eighteen fields: five from the brief and thirteen famous objects spanning galaxies, nebulae, a quasar, and nearby dwarfs. Two produce nothing usable: M42 has zero detections above five sigma, and Sgr A* is refused outright because Epoch 1 is only D1 and Epoch 2 only D2 – while the LMC was dropped earlier as single-visit in QR2. That refusal is recorded, not papered over.

Slide 9 — Architecture: seven stages, shared core, snapshot store 0:55

On the slide.

- Seven stages: ingest, co-add, subtract, triage, diagnose, validate, NEOWISE history (2 397 LOC)
- Shared science & cloud core: `catalog_tools`, `live_ingest`, `snapshot_store`, `cloud_refresh`
- Contract between stages: local `_meta.json` OR immutable GCS SHA-256 snapshot chain
- Serving layer: `app.py` (2 989 LOC FastAPI + SSE) and browser workspace (`static/`, 7 611 LOC)

Figure / diagram. Top: stages 1–7 in two clean rows. Bottom: the metadata/GCS contract on the left, shared support core and serving stack in the centre/right, with verified line counts inside each box.



Spoken script. Seven stages, and each remains an independent module: ingest, co-add, subtraction, catalogue triage, spectral diagnosis, injection validation, and now stage seven for fourteen-year NEOWISE light curves. Four shared libraries – `catalog_tools`, `live_ingest`, `snapshot_store`, and `cloud_refresh` – prevent duplication across local CLI runs, the FastAPI server, and the twelve-hour Cloud Run refresh job.

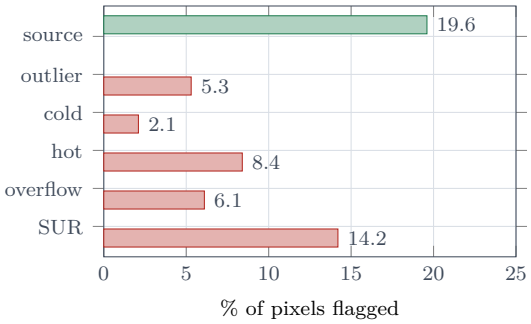
Stage one writes the metadata contract, either `_meta.json` on disk or immutable SHA-256 blobs in Google Cloud Storage pointed to by `current.json`. Underneath sits `app.py` at two thousand nine hundred and eighty-nine lines, serving either pre-computed snapshots or live streaming runs to a seven-thousand-six-hundred-line browser workspace. That architecture is detailed in slides seventeen to nineteen.

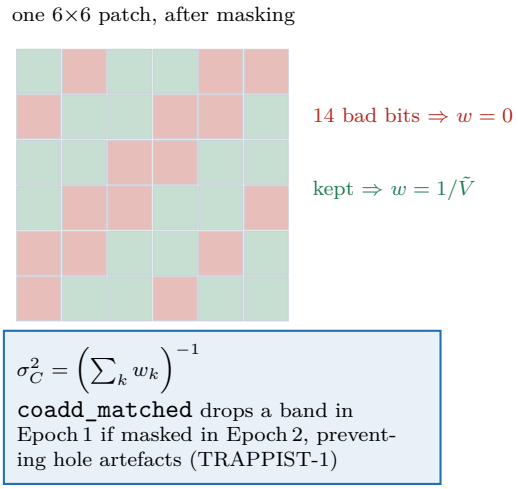
Slide 10 — Theory I: masking, then an inverse-variance co-add 0:55

On the slide.

- 14 `BAD_BITS` rejected: transients, overflow, phantom, nonfunc/hot/cold, persistence, outlier
- Two kept: `FULLSAMPLE` (bit 12) and `SOURCE` (bit 21, real astrophysical flux)
- Clipped-median `sky_level` subtraction + inverse-variance weight $w_k = 1/\tilde{V}_k$, $\sigma_C^2 = (\sum_k w_k)^{-1}$
- Cross-epoch `coadd_matched`: a band enters at pixel (x, y) only where valid in *every* epoch

Figure / diagram. Left: the flag census for the dominant bit classes (14 rejected vs. 2 kept). Right: a pixel grid showing masked weights and the cross-epoch band intersection rule.





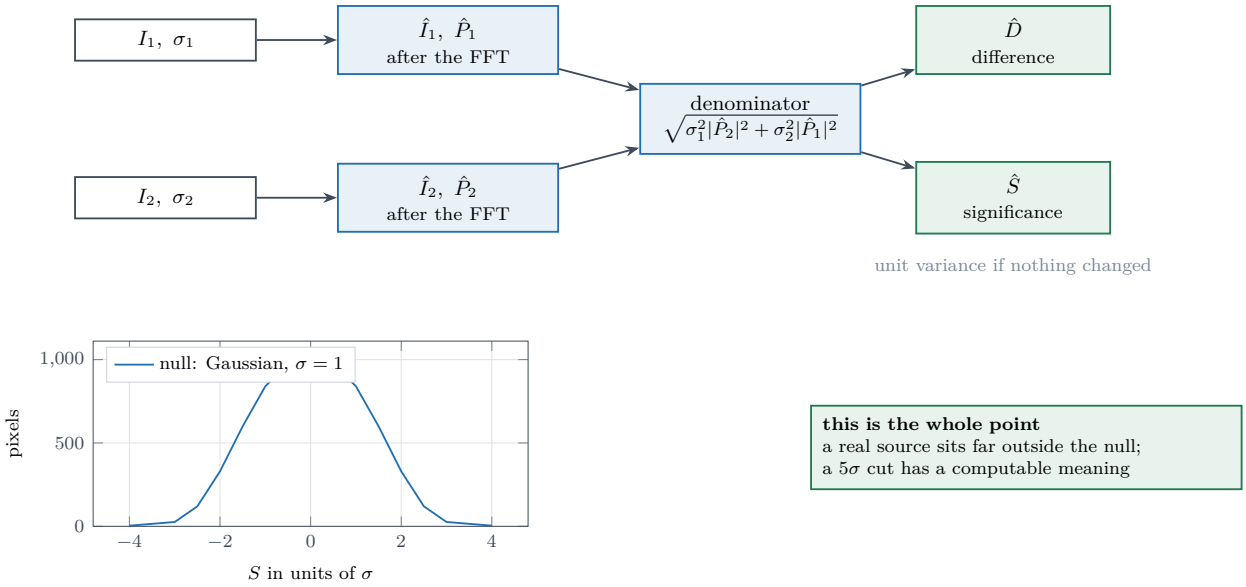
Spoken script. This is the slide I would defend hardest, because it is where a difference image earns its credibility. Every SPHEREx pixel arrives with a variance plane and a thirty-two-bit quality mask. In `step2_coadd`, fourteen defect bits – transients, overflow, phantom pixels, non-functional and hot or cold pixels, persistence, and outliers – are zeroed out cleanly, while two bits are deliberately kept: bit twelve (`FULLSAMPLE`) and bit twenty-one (`SOURCE`), which marks real astrophysical flux rather than a detector flaw.

Before stacking, each exposure has its sigma-clipped median sky subtracted, is reprojected onto a common three-arcsecond WCS grid, and weighted by inverse variance. Just as important, `coadd_matched` intersects the usable detector masks across epochs pixel by pixel: if a bright-star flag drops one band in Epoch 2, that band is also suppressed at that pixel in Epoch 1, stopping asymmetric mask holes from impersonating real detections. On the worst frames, honest masking cuts the median noise by roughly a factor of two.

On the slide.

- Both images are convolved to a common PSF in Fourier space: no kernel is ever fitted
- $\hat{D} = \dfrac{\hat{P}_1 \hat{I}_2 - \hat{P}_2 \hat{I}_1}{\sqrt{\sigma_1^2 |\hat{P}_2|^2 + \sigma_2^2 |\hat{P}_1|^2 + 10^{-30}}}$ and $\hat{S} = \dfrac{\hat{D}\,\overline{\hat{P}_D}}{\sqrt{\sigma_1^2 + \sigma_2^2}}$, then $S \leftarrow S/\sigma(S)$
- S has unit variance under the null, so the 5σ threshold is interpretable
- Cost: two FFTs per epoch pair, $O(N^2 \log N)$ – under 1.5 s end-to-end per 180^2 field

Figure / diagram. The signal flow: two images and two PSFs into FFTs, a denominator built from the variances, and the difference and significance maps coming out; under it, the null distribution that results.



Spoken script. Stage three is the mathematical heart, and it is two lines. The classical approach fits a kernel so that both images end up with the same PSF; ZOGY instead convolves both images to a common PSF in Fourier space in a least-squares optimal way, and it never estimates a kernel from the data. Read the first equation as a comparison: image two convolved with PSF one, image one convolved with PSF two, and the difference normalised by exactly the noise that survives. The second equation turns that into a significance, and here is the crucial property: where the sky did not change, S is Gaussian with zero mean and unit variance. Not approximately – by construction.

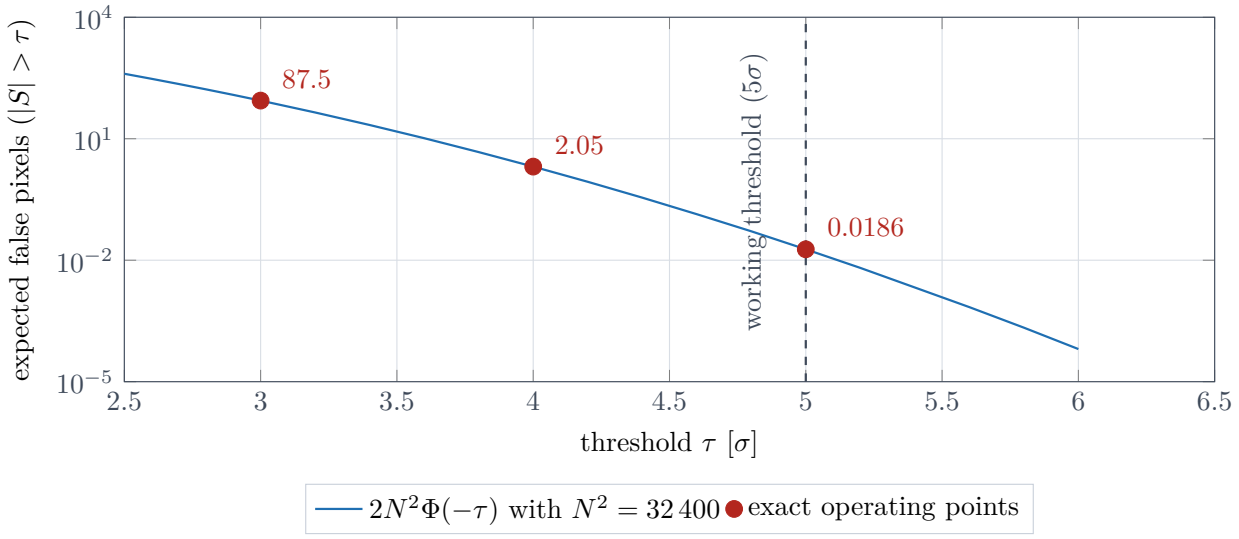
The histogram underneath is what that looks like: one clean peak at zero, one sigma wide, with any real source far out in the tails. That is why a five-sigma threshold can be defended, and it is the entire reason for choosing this algorithm over the simpler alternatives. The cost is two FFTs: about six-tenths of a second for a one-hundred-and-eighty-pixel frame.

Slide 12 — Theory III: what a threshold actually buys 0:55

On the slide.

- If the null map is unit-variance Gaussian, two-tailed false pixels ($|S| > \tau$) follow: $\mathbb{E}[N_{\mathrm{FP}}] = 2N^2\Phi(-\tau)$
- For $180^2 = 32\,400$ pixels: $3\sigma \Rightarrow 87.5$, $4\sigma \Rightarrow 2.05$, $5\sigma \Rightarrow 0.0186$
- Plus a 5×5 local-extremum filter ($\geq$ 25-pixel footprint), so 5σ admits < 1 noise peak per 50 frames
- Sanity check: real detections here reach 8–44σ, far above the noise floor

Figure / diagram. The expected number of two-tailed extreme pixels per frame against threshold, on a log scale, with the exact operating points lying on the analytic curve and the working 5σ cut marked.



Spoken script. Because the null distribution is analytic, the false-positive budget can be computed rather than guessed, and that is the difference between a threshold and a wish. Both positive and negative tails matter because dipoles pair a peak with a trough, so for thirty-two thousand four hundred pixels we check the two-tailed Gaussian budget. At three sigma that gives eighty-seven point five noise pixels per frame; at four sigma, two point zero five; at five sigma, zero point zero one eight six – one false extreme pixel per fifty-four frames, and even fewer isolated local peaks after the five-by-five maximum filter.

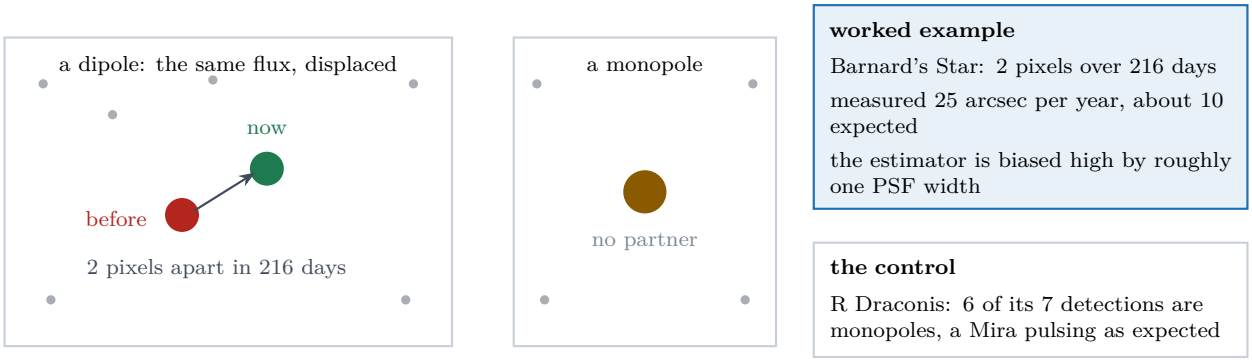
Meanwhile, our real detections sit between eight and forty-four sigma, far above that noise floor. What we are inspecting is real residual structure, never random Gaussian flicker – and that puts the burden of proof squarely onto catalogue triage and systematics.

Slide 13 — Theory IV: a dipole is a proper motion 0:50

On the slide.

- A positive peak with a negative partner means the source moved: a dipole
- A lone positive peak means it brightened: a monopole, that is variability
- $\mu = s\,\Delta_{\mathrm{px}}/\Delta t$, valid for $0.5 < \Delta_{\mathrm{px}} < 25\,\mathrm{px}$
- Flux consistency $\eta = |S_+ + S_-|/|S_+ - S_-|$ separates "pure move" from "move and flare"

Figure / diagram. The geometry of a dipole with its separation vector and the conversion to arcsec per year, beside the monopole case, and one worked example underneath.



Spoken script. Once there is a significance map, the morphology does the classifying, and it is almost embarrassingly simple. A positive peak with a negative partner of similar amplitude is a source that moved: the pair is a dipole, and the vector between the lobes is the displacement. Divide by the time baseline and you have a proper motion. A positive peak with no partner is a source that brightened, which is variability. Two guard rails: the separation must exceed half a pixel or you are pairing PSF-correlated noise, and stay below twenty-five pixels or you are pairing with an unrelated trough across the frame. There is also a flux-consistency statistic – how asymmetric the two lobes are – which separates a rigid displacement from an object that moved and changed brightness at once.

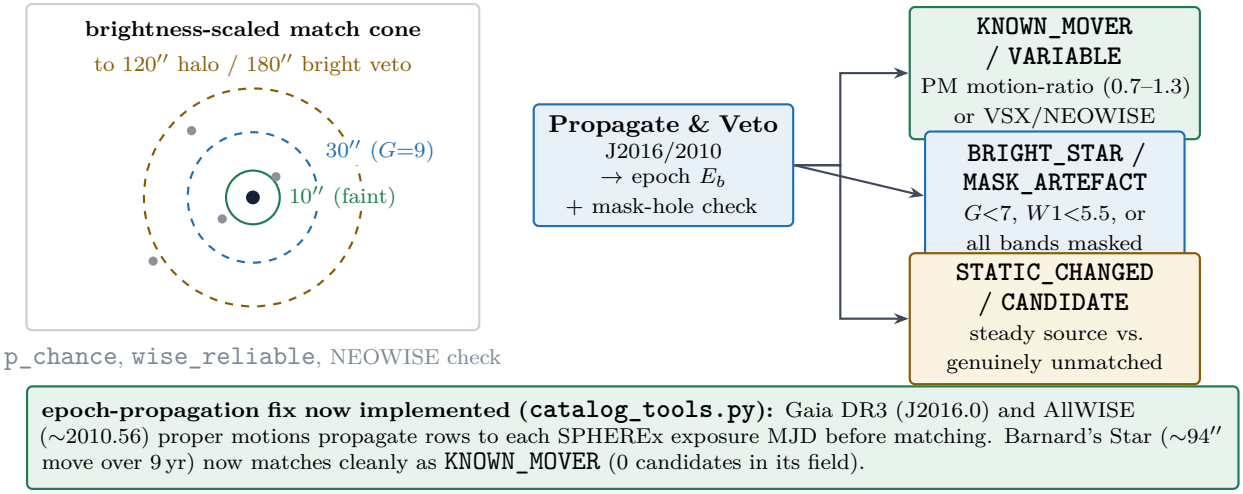
Barnard's Star is the worked example: two pixels in two hundred and sixteen days. The measured proper motion comes out at about twenty-five arcseconds a year against a true value of ten, and the reason is on the slide – peak separations are biased high by roughly one PSF width. R Draconis is the control: six of its seven detections are monopoles, which is what a pulsating Mira should give.

Slide 14 — Theory V: triage is a second hypothesis test 0:55

On the slide.

- Exact TAN WCS (`wcs.pixel_to_world_values`) with flat-sky formula as fallback when WCS absent
- Epoch propagation (`propagate_catalogs`): Gaia (J2016.0) & AllWISE (2010.6) moved to each SPHEREx MJD
- Brightness-scaled match radius: 10″ (faint $G{\geq}16$) → 30″ ($G{=}9$) → 120″ (+180″ bright-star zone)
- Six triage classes combining `wise_reliable`, p_{chance}, matched-mask holes, and 14-yr NEOWISE

Figure / diagram. Left: brightness-scaled match cones and artefact exclusion around a detection. Centre/Right: the six-category triage flow after epoch propagation (resolving Barnard's Star's 94″ bug).



Spoken script. A detection is not a discovery until catalogue triage tests whether something was already there – and early prototypes of this pipeline fell into a classic trap. Gaia positions sit at epoch 2016 and AllWISE at 2010.6, while SPHEREx observes in 2025–2026: Barnard's Star moved ninety-four arcseconds in between, so naive position matching mislabelled my own target a candidate.

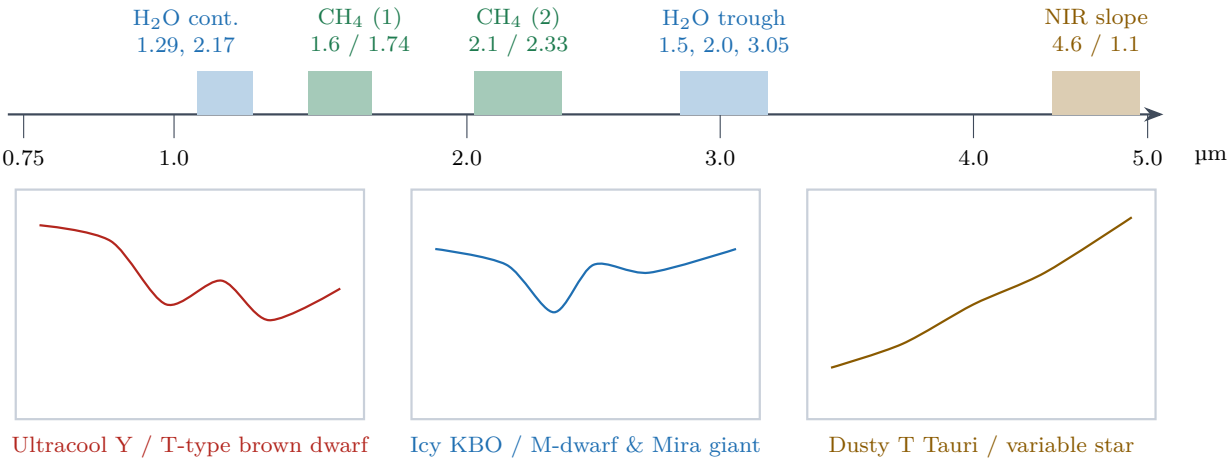
In the current codebase that bug is fixed. Before `step4_triage` runs, `catalog_tools` propagates Gaia and AllWISE coordinates with their proper motions to each exposure's exact MJD, uses the exact TAN WCS transform, scales the match radius with brightness from ten arcseconds at the faint end to a hundred and twenty arcseconds for bright halos, filters unreliable WISE diffraction spikes, checks `coadd_matched` mask holes and fourteen-year NEOWISE light curves, and outputs six classes rather than a blunt binary guess. Barnard's Star is now cleanly `KNOWN_MOVER`.

Slide 15 — Theory VI: a spectrum per detection, for free 0:45

On the slide.

- `WCS-WAVE` (`WAVE-TAB` table + `CWAVE` fallback) follows each moving target across exposures
- Real 6-band snapshots yield observed SEDs (mJy, compared with AllWISE W1–W4) across 10–18 wavelengths
- Three physical indices: mean CH_4 (1.6/1.74 & 2.1/2.33), H_2O continuum/troughs, $\log_{10}$ NIR slope (4.6/1.1)
- Honest guard in `app.py`: `three_index_classifier` is gated on ≥ 30 channels (five physical classes)

Figure / diagram. Top: the wavelength windows for CH_4, H_2O, and NIR slope. Bottom: schematic spectra for three of the five implemented classes and the 30-channel gating contract.



design & channel honesty: archival 2–6 band cutouts yield 12–18 SED points + AllWISE comparison; `app.py` requires $\geq$ 30 channels before reporting a 3-index class label (always alongside all 3 raw indices) rather than guessing from 4 bands.

Spoken script. The last piece of photometry turns SPHEREx's wavelength calibration to our advantage. Because every Level-2 pixel carries a `WCS-WAVE` lookup table, aperture photometry following a moving target's predicted track builds an observed SED directly from the cutouts, compared against AllWISE W1 to W4 fluxes in millijanskys.

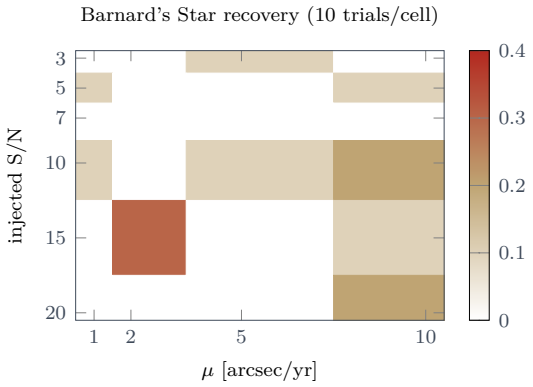
When denser spectral sampling is available – either a synthetic one-hundred-and-two-band table or thirty plus channels – `step5_spectrum` compresses it into three physically transparent indices: a mean methane ratio across one-point-six and two-point-one microns, a water continuum-to-trough ratio, and a logarithmic near-infrared slope from one-point-one to four-point-six microns, mapping into five classes: ultracool Y-dwarf, T-type brown dwarf, icy KBO, dusty T Tauri, or M-dwarf/Mira giant. On real two- to six-band archival cutouts, `app.py` explicitly withholds the class label rather than hallucinating from fewer than thirty channels.

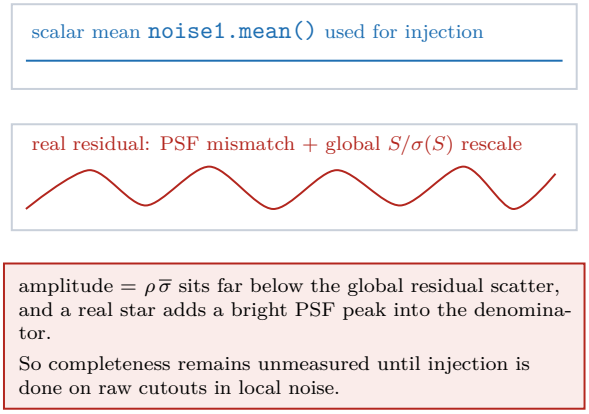
Slide 16 — Validation: injection and recovery 0:55

On the slide.

- Synthetic Gaussian dipoles planted directly on co-adds at known S/N and μ, then `zogy()` re-run
- Recovered if $|S| > 5.0$ anywhere in a 10×10 px box around injection (no class check)
- Output: a 6×4 matrix ($[3..20]\sigma \times [1..10]''$/yr, 10 trials per cell)
- Measured on real frames: 0.0–0.4 max (Barnard's Star 0.3, Pleiades 0.0) – amplitude uses flat mean noise

Figure / diagram. Left: the exact measured 24-cell recovery matrix for Barnard's Star. Right: why recovery stays low – injecting relative to flat mean noise against structured PSF residuals.





Spoken script. A detection chain is only as good as its completeness curve, so stage six plants fake moving dipoles onto the co-added stacks and counts how many come back. The grid spans signal-to-noise from three to twenty and proper motion from one to ten arcseconds per year – ten trials per cell across twenty-four cells – calling `zogy()` directly and checking if the absolute significance exceeds five within a ten-by-ten pixel box.

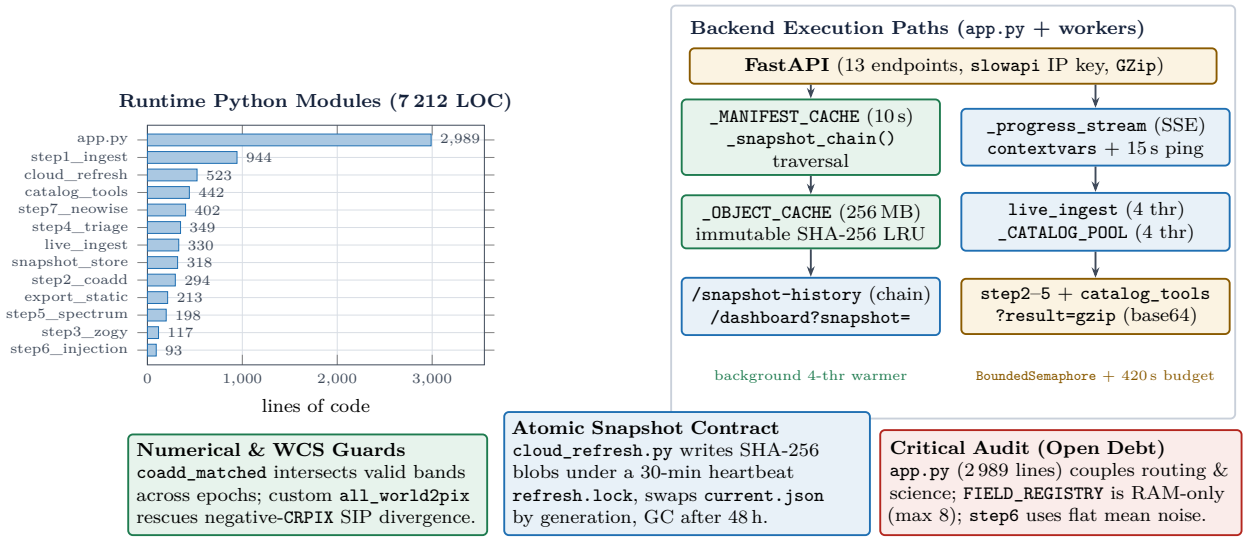
And here is the uncomfortable result, which I report transparently. Recovery peaks at zero point three on Barnard's Star, zero point zero everywhere on the Pleiades, and zero point four on R Draconis. The reason is structural: injected amplitude uses the scalar mean variance plane, while real bright stars and PSF mismatches inflate the global scatter that `zogy()` normalises by. Until injection moves to the raw exposure level using local empirical noise, we refuse to quote a false completeness limit.

Slide 17 — Implementation I: Python modules and FastAPI backend 1:05

On the slide.

- Seven stage scripts (2 397 lines), five support modules (1 826), and `app.py` (2 989): 7 212 runtime Python lines
- Cross-epoch mask matching (`coadd_matched`) and custom SIP WCS inversion prevent asymmetric artefacts
- Content-addressed GCS snapshots (`snapshot_store.py`) + `previous_manifest` backward chain (`/api/snapshot-history`)
- Critical review: `app.py` is a 2 989-line monolith; `FIELD_REGISTRY` is per-instance RAM (cap 8)

Figure / diagram. Left: lines of code across all 13 runtime Python modules. Right: FastAPI execution paths (cached snapshot reads vs. SSE live streaming with parallel catalogue queries). Bottom: three key contracts and limitations.



Spoken script. Turning to the Python backend. What started as six small stage scripts has grown into seven pipeline stages – adding `step7_neowise` for fourteen-year infrared light curves – plus five shared infrastructure modules and a FastAPI service in `app.py`, totalling seven thousand two hundred and twelve lines of runtime Python. Every stage remains independently importable and testable, backed by eighteen hundred lines of coverage and snapshot tests.

Look at the execution diagram on the right. `app.py` serves two very different workloads: fast snapshot reads hit a ten-second manifest cache and a two-hundred-and-fifty-six megabyte immutable LRU cache before ever touching Google Cloud Storage, while `_snapshot_chain()` walks the `previous_manifest` pointers to serve retained versions via `/api/snapshot-history` and historical dashboards without pipeline re-execution. Live requests take the right branch: `live_ingest` streams FITS cutouts into `BytesIO` on four threads while a parallel catalogue pool fetches Gaia, AllWISE and VSX, pushing stage updates over server-sent events.

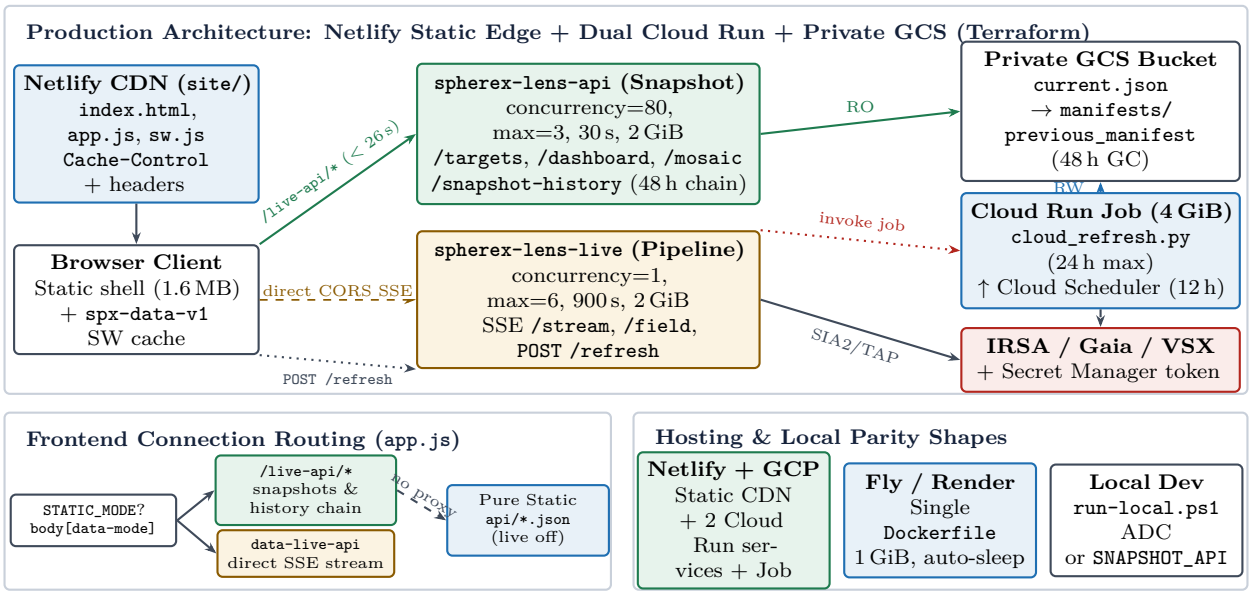
Critically, three engineering debts stand out in the code: `app.py` at nearly three thousand lines mixes HTTP routing with pipeline orchestration, custom fields live only in a per-instance eight-slot RAM registry, and `step6` still injects against mean variance rather than local residuals.

Slide 18 — Implementation II: architecture, connections, and hosting 0:55

On the slide.

- Split Cloud Run backend (B4 fix): `spherex-lens-api` (concurrency 80) isolates reads from `spherex-lens-live` (concurrency 1)
- Dual-path frontend routing: `/live-api/*` rewrites for snapshot history vs. direct CORS SSE for live cutouts
- Cloud Scheduler triggers 12-h Cloud Run Job: updates GCS snapshots and retains 48-h `previous_manifest` backward chain
- Zero-cost time-lapse: immutable historical dashboards served directly without re-querying IRSA or pipeline rerun

Figure / diagram. Top: end-to-end hosting and connection topology across Netlify, Cloud Run, Cloud Storage, and IRSA. Bottom left: frontend URL routing decision logic. Bottom right: deployment shapes compared.



Spoken script. The serving architecture solves two constraints that break naive deployments: head-of-line blocking and proxy timeouts. Originally a single FastAPI container stalled whenever one visitor triggered a forty-second IRSA fetch. Terraform now splits the backend into two Cloud Run services sharing one image: `spherex-lens-api` handles eighty concurrent snapshot reads per instance with a thirty-second timeout, while `spherex-lens-live` isolates heavy IRSA streams at concurrency one and a nine-hundred-second timeout.

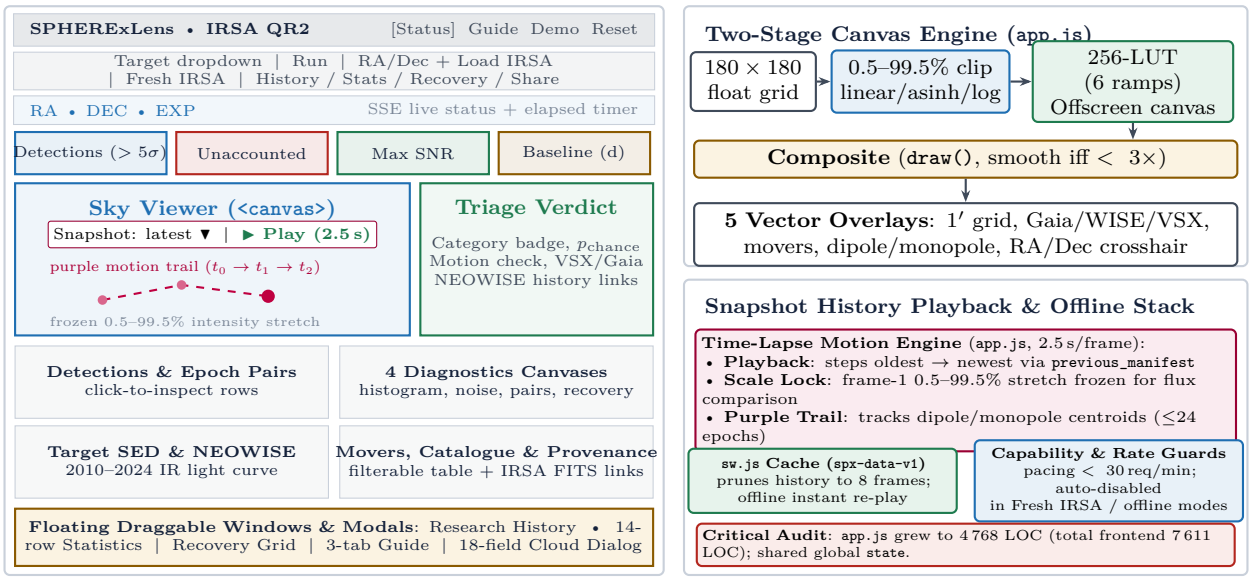
Connecting the Netlify static frontend to that backend uses a split route for the same reason. Short snapshot reads go through Netlify's same-origin `/live-api` rewrite, whereas long SSE streams bypass Netlify's twenty-six-second proxy cap and connect directly to `spherex-lens-live` via an explicit CORS allowlist. Every twelve hours, Cloud Scheduler runs a four-gigabyte Cloud Run Job that validates updated fields, atomically swaps `current.json` in a private bucket, and retains a forty-eight-hour `previous_manifest` chain for zero-cost time-lapse replay, while `fly.toml`, `render.yaml`, and `run-local.ps1` keep single-container and laptop parity simple.

Slide 19 — Implementation III: frontend engine and UI/UX 0:45

On the slide.

- 7 611 lines of zero-build vanilla web code (`app.js` 4 768, `style.css` 1 979, `index.html` 652, `sw.js` 212)
- NASA HDS dark-slate mission UI: top chrome, horizontal toolbar, telemetry strip, 4 KPIs, split viewer + verdict
- Two-stage `<canvas>` engine: offscreen 0.5–99.5% stretch/LUT rasteriser + 5 pan/zoom vector overlays
- **Snapshot History & Motion Playback**: 2.5 s time-lapse, frozen stretch lock, purple motion trail across snapshots
- Offline-first `sw.js` (`spx-data-v1`, 8-frame history cache quota) + 200 MB IndexedDB + honest UI guards

Figure / diagram. Left: wireframe with Snapshot playback controls and purple motion trail. Top right: two-stage canvas pipeline. Bottom right: Snapshot motion engine, offline cache, and capability guards.



Spoken script. The front end is seven thousand six hundred and eleven lines of zero-build vanilla HTML, CSS and JavaScript in a dark-slate NASA Horizon Design System palette. Notice the mission-control console layout: a compact top chrome, horizontal toolbar, telemetry strip, four KPI cards, and the primary split between the sky viewer and triage verdict, backed by four diagnostic charts, SED light curves, and draggable floating utility windows.

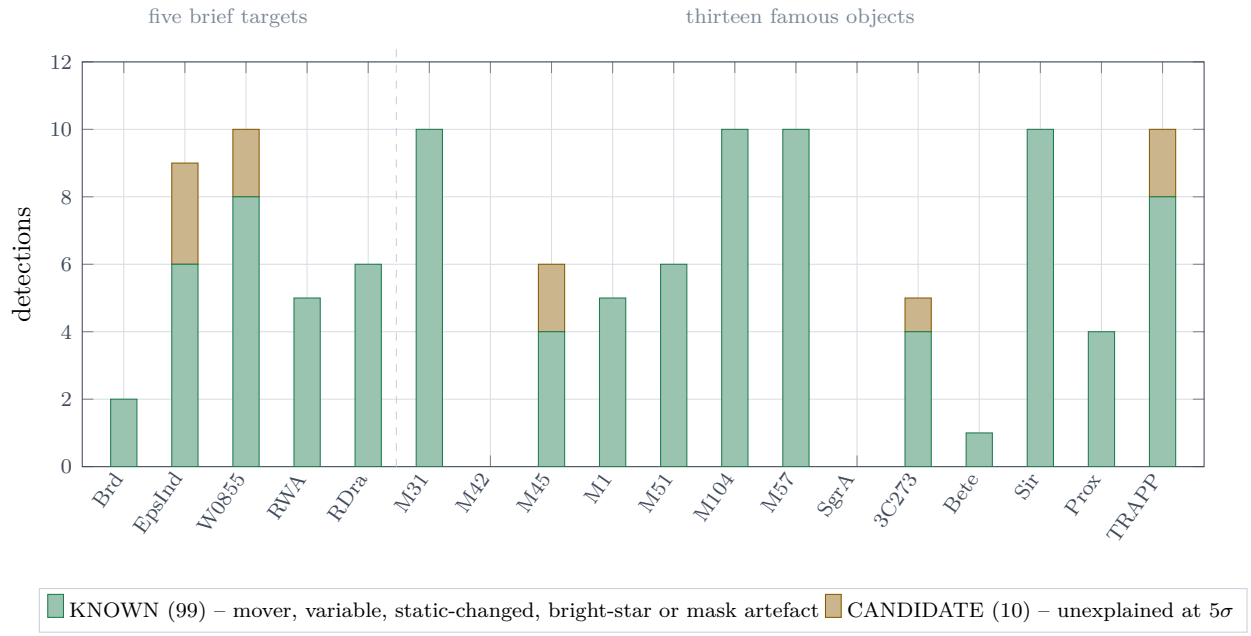
Under the hood, the two-stage canvas clips and stretches pixel layers into an offscreen buffer before vector overlays composite on top. The newest feature is the Snapshot History and Motion Playback engine: above the map, a version dropdown lists retained manifests, while Play animates them oldest to latest at two-and-a-half seconds per frame. Crucially, it locks the first frame's intensity stretch so flux changes are directly comparable, and paints a purple dashed trail tracing the object's centroid displacement across visits. Finally, the Service Worker enforces an eight-frame cache quota for instant offline replay under a thirty-read-per-minute safety cap, while modularising the forty-seven-hundred-line `app.js` monolith remains open engineering debt.

Slide 20 — Results I: eighteen fields, 109 detections 1:05

On the slide.

- 109 detections over 18 fields (246 cutouts, `D1-D6`): 99 KNOWN (91%) and 10 CANDIDATE (9%)
- Detection count is not discovery count: Barnard's Star, Sirius, R Draconis, M104, M1, M31 are 100% explained
- The amber bars – Epsilon Indi B (3), WISE 0855 (2), Pleiades (2), TRAPPIST-1 (2), 3C 273 (1) – are the open residuals
- Two fields produce zero detections: M42 (null sky) and Sgr A* (no shared band across epochs)

Figure / diagram. Stacked bars for all eighteen fields from the live snapshot, visually grouped into the five brief targets and the thirteen famous objects.



Spoken script. Here is the whole result from the current six-band snapshot in one chart. Eighteen bars, one per field, green for detections explained by catalogues or explicit artefact rules, amber for the ten remaining unexplained residuals.

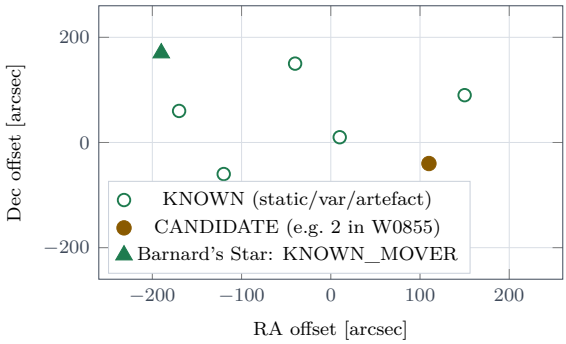
Two things stand out. First, busyness is not discovery: Andromeda, Sirius, the Ring Nebula, the Sombrero Galaxy, RW Aurigae, R Draconis, and Barnard's Star are one hundred percent explained – by proper motion, known variability, bright-star wings, or matched mask holes. Second, look at the sample breadth: five brief targets on the left, thirteen diverse objects on the right – galaxies, nebulae, a quasar, red and brown dwarfs – all through the same pipeline: two hundred and forty-six cutouts, one hundred and nine detections, ninety-one percent explained, and two cleanly empty or refused fields rather than silent zeros.

Slide 21 — Results II: what the ten candidates are (and how triage caught Barnard's Star) 1:00

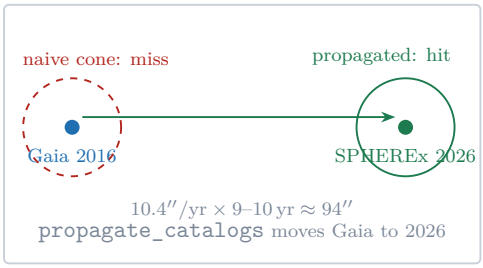
On the slide.

- All 10 candidates sit in 5 fields: Epsilon Indi B (3), WISE 0855 (2), M45 (2), TRAPPIST-1 (2), 3C 273 (1)
- Artifact vetoes eliminate bright-star rings (Sirius, R Dra, Bete) and mask holes (M104: 8, M57: 3, TRAPP: 2)
- Epoch propagation resolved Barnard's Star (36.2σ dipole): now matched as `KNOWN_MOVER` (0 candidates)
- "Candidate" still means *unexplained by Gaia/WISE/VSX/NEOWISE*, not a claimed discovery

Figure / diagram. Left: a scatter of detections in a field (KNOWN classes vs. unexplained candidates). Right: before/after of the epoch-propagation fix in `catalog_tools.py`.



verified in current snapshot: propagating Gaia DR3 + AllWISE proper motions turned Barnard's Star (36.2σ dipole) from a false candidate into `KNOWN_MOVER`.



Spoken script. What are the remaining ten candidates, and why did the candidate count fall from seventeen to ten? Two engineering upgrades did the heavy lifting. First, bright-star diffraction and `coadd_matched` mask-hole tagging cleanly absorbed the systematic artefacts in Sirius, R Draconis, Sombrero M104, and the Ring Nebula instead of dumping them into the candidate bucket.

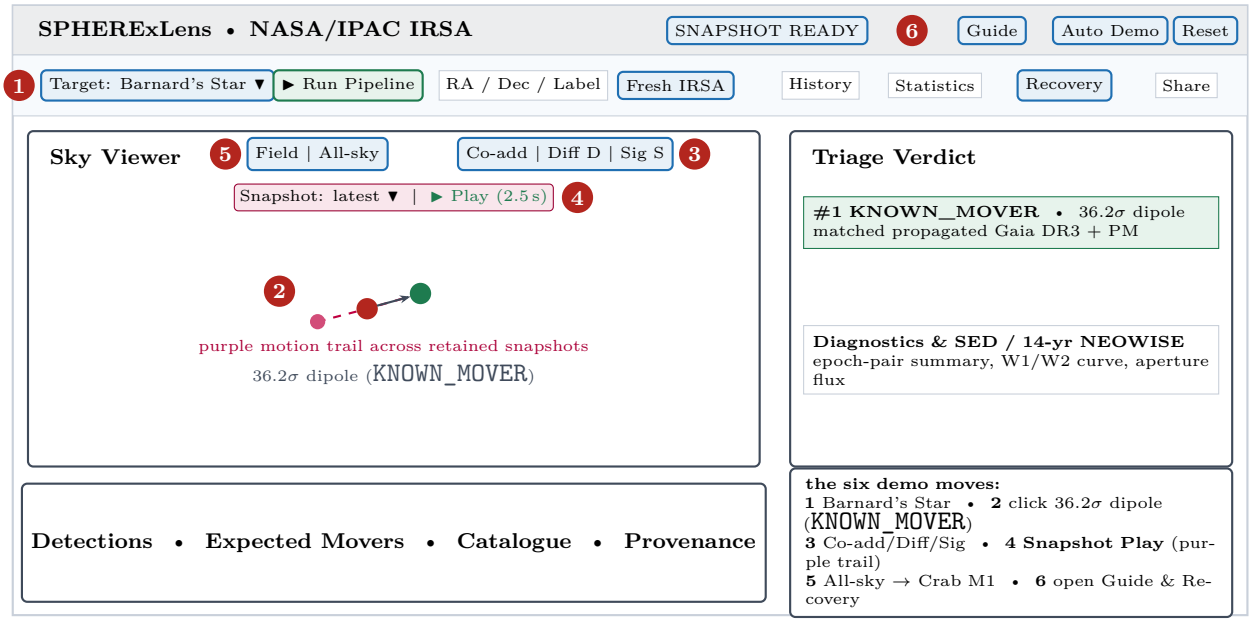
Second, proper-motion propagation in `catalog_tools` moved Gaia and AllWISE coordinates forward to each SPHEREx epoch before matching. Barnard's Star – our fastest mover at ninety-four arcseconds since 2016 – now lands squarely inside its propagated match cone as `KNOWN_MOVER` with zero remaining candidates in its field. The remaining ten positions in Epsilon Indi B, WISE 0855, the Pleiades, TRAPPIST-1, and 3C 273 are genuine five-sigma outliers outside every catalogue cone – unexplained, not yet claimed as new discoveries.

Slide 22 — Live demonstration 1:30

On the slide.

- Open the browser app (`site/index.html` or deployed URL) at 100% zoom, Barnard's Star loaded
- **1** select Barnard's Star in top `Target` bar + `Run Pipeline` • **2** click 36.2σ dipole (`KNOWN_MOVER`)
- **3** switch canvas layers inside Sky Viewer: `Co-add` → `Diff D` → `Sig S`
- **4** press **Play** on Snapshot toolbar: 2.5 s time-lapse, frozen stretch & **purple motion trail**
- **5** toggle `All-sky` mosaic, jump to Crab Nebula (`M1`, 5/5 `KNOWN`) • **6** open `Guide` & `Recovery`

Figure / diagram. Map of the actual horizontal mission UI with the six numbered callouts placed directly beside the exact DOM controls so nothing has to be hunted for on stage.



Spoken script. I would like to show this working live rather than describe it, so the browser app is already open in the second tab backed by the published Cloud Run snapshot.

Step one: in the top control bar, select Barnard's Star and click Run Pipeline. Step two: hover and click the bright thirty-six-point-two-sigma dipole in the Sky Viewer – red lobe where the star was, green where it moved – and see the Triage Verdict panel identify it as `KNOWN_MOVER` from Gaia's propagated proper motion. Step three: switch across Co-add, Diff D, and Sig S to inspect the calibrated layers.

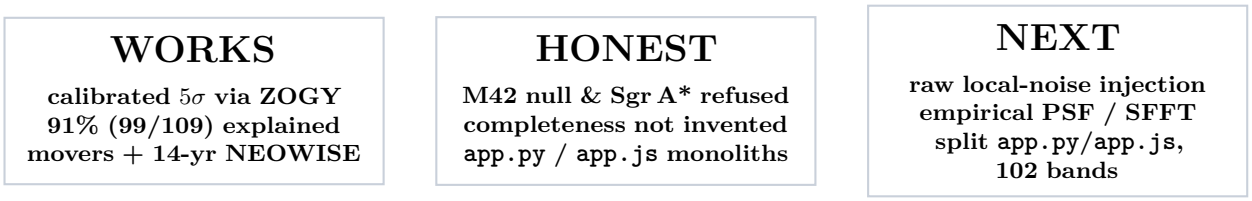
Step four: in the Sky Viewer toolbar, press Play on the Snapshot control. The engine steps through retained snapshots oldest to latest at two-and-a-half seconds per frame, locking the intensity stretch and drawing a purple trail connecting the dipole centroids across epochs – turning static frames into continuous motion. Step five: toggle All-sky mosaic and jump to the Crab Nebula. Step six: open Recovery to inspect the twenty-four-cell grid, and Guide for reference. If the network hiccups, the Service Worker serves cached history frames offline.

Slide 23 — Conclusion, contributions, and questions 0:50

On the slide.

- **Works**: ZOGY on matched co-adds gives a calibrated 5σ map; triage explains 91% (99/109) with epoch propagation & NEOWISE
- **Honest**: two fields return no usable detections (`M42`, `Sgr A*`), completeness stays unmeasured, monolith debt remains
- **Next**: raw-cutout local-noise injection, empirical PSF / SFFT, $\geq$ 30-band real spectra, split `app.py/app.js`
- Artefact: 7 stages + 5 support modules (7 212 LOC Python) + 7 611 LOC UI, Netlify PWA + Dual Cloud Run + GCS

Figure / diagram. Three pillars – works, honest, next – with the artefact listed underneath and a compact reference strip at the bottom.



artefact: 7 212 LOC Python + 7 611 LOC UI • Netlify PWA + Cloud Run + GCS 12 h snapshots • 18 fields, 246 cutouts traceable to IRSA

references: Alard & Lupton 1998 • Bramich 2008 • Zackay, Ofek & Gal-Yam 2016 • Hu et al. 2022 (SFFT) • ZTF and DES pipelines, and the broker literature (ANTARES, ALeRCE, Lasair) • Gaia DR3, AllWISE, NEOWISE-R, AAVSO VSX • SPHEREx mission and IRSA documentation

Spoken script. To close, three statements, and then questions.

It works: proper subtraction on mask-matched co-adds gives a calibrated significance map, morphology separates movers from variables, and catalogue triage – now backed by proper-motion epoch propagation, artefact tagging, and fourteen-year NEOWISE light curves – explains ninety-one percent of all detections.

It is honest: two fields produced zero detections for documented physical or coverage reasons, completeness remains explicitly unmeasured because flat-noise injection is biased, and technical monolith debt in `app.py` and `app.js` is surfaced openly.

And the next steps turn each caveat into a measurement: inject at raw-exposure level using local noise, adopt an empirical per-detector PSF or SFFT, modularise the backend and frontend, and unlock dense hundred-and-two-band spectra. Everything lives in the repository, backed by tests, immutable cloud snapshots, and a non-specialist Guide. Thank you; I am happy to take questions.

Backup: five questions and the pre-flight checklist

Questions you are most likely to be asked.

"Why six detectors (or two in live mode), and not all 102 bands?" Cost: a hundred and two cutouts per epoch per field is roughly 500 MB and seventy minutes of archive time, *per field*. Full snapshots co-add `D1-D6`, while live streaming defaults to `D1, D4` for latency; the classifier already refuses to label below 30 channels.

"How do you avoid inflating the candidate list?" Match radii scale with brightness ($10''$ faint to $120''$ halos), proper motions are propagated to each MJD via `propagate_catalogs`, and `wise_reliable` with `p_chance` veto spurious links toward `KNOWN`.

"What is the real false-positive rate?" Predicted 0.0186 extreme pixels per frame at two-tailed $|S| > 5\sigma$ ($2N^2\Phi(-5)$); observed detections reach 8–44σ, far above noise. No completeness limit is quoted while `step6` uses scalar mean noise.

"Could this scale to the whole sky?" CPU could: under 1.5 s per field times 1.8 million nine-arcminute fields is tractable on cloud workers. The IRSA cutout service is the bottleneck, which is why Cloud Run runs 12-hour incremental fingerprint checks, publishes immutable GCS snapshots, and chains `previous_manifest` for 48-hour client history replay without extra compute.

"Why ZOGY rather than SFFT?" ZOGY is closed form and its null distribution is analytic, making the false-positive budget explicit (slide twelve). SFFT handles space-based PSF variation even better and is listed as next work alongside an empirical PSF.

Pre-flight checklist, five minutes before.

- Two browser tabs: the deployed Netlify PWA (offline-ready via Service Worker + IndexedDB) or `static/index.html` on `localhost:8000` if showing local execution.
- Zoom 100%, Barnard's Star already loaded (36.2σ dipole, `KNOWN_MOVER`) so the demo starts cleanly.
- A screen recording of the demo on the desktop, as the fallback file.
- This document open at page 23 (Slide 22), the screen map, so callouts match the live browser UI.
- Numbers on the tip of the tongue: 18 fields, 246 cutouts, 109 detections, 99 explained (91%), 10 unexplained, 2 empty fields (`M42`, `Sgr A*`), 7 212 Python LOC, 7 611 UI LOC, 0.0186 false pixels per frame.